

Concurrent Programming - Exercícios 3

CCS

1. Which of the following expressions of CCS are correct?.

- (a) $(a.0 + \bar{a}.A) \setminus \{a, b\}$
- (b) $(a.0 + \bar{a}.A) \setminus \{a, \tau\}$
- (c) $\tau.\tau.B + 0$
- (d) $(a.b.B + \bar{a}.0) | B$
- (e) $(a.b.B + \bar{a}.0).B$

2. Let $A := b.a.B$, using the inference rules show that the following transitions exist:

$$\bullet (A | \bar{b}.0) \setminus \{b\} \xrightarrow{\tau} (a.B | 0) \setminus \{b\}$$

3. Solve the *CCS* exercises in PseuCo.com

4. Consider the following definitions of a researcher that takes coffee and publishes articles.

$$\begin{aligned} CM &:= coin?.coffee!.CM \\ CS &:= pub!.coin!.coffee?.CS \\ Uni &:= (CM | CS) \setminus \{coin, coffee\} \end{aligned}$$

Use the CSS inference rules to obtain the reachable fragment of $\llbracket Uni \rrbracket_{\Gamma}$. Test in pseuco.com. Compare with $Spec := pub!.Spec$.

5. Let $A := (a.A) \setminus \{b\}$ show that the reachable fragment of $\llbracket A \rrbracket_{\Gamma}$ is infinite.

6. The following definitions try to solve the mutual exclusion with a semaphore.

(a)

$$\begin{aligned} Mutex_1 &:= (User | Sem) \setminus \{p, v\} \\ User &:= \bar{p}.enter.exit.\bar{v}.User \\ Sem &:= p.v.Sem \end{aligned}$$

Use the CCS inference rules to obtain the reachable fragment of $\llbracket Mutex_1 \rrbracket_{\Gamma}$. Test in pseuco.com.

(b) Let

$$Mutex_2 := ((User | Sem) | User) \setminus \{p, v\}$$

Use the CCS inference rules to obtain the reachable fragment of $\llbracket Mutex_2 \rrbracket_{\Gamma}$. Test in pseuco.com. There will be changes if one uses $User := \bar{p}.enter.\bar{v}.exit.User$?

(c) Let

$$\begin{aligned} FMutex &:= ((User | Sem) | FUser) \setminus \{p, v\} \\ FUser &:= \bar{p}.enter.(exit.\bar{v}.FUser + exit.\bar{v}.0) \end{aligned}$$

Use the CCS inference rules to obtain the reachable fragment of $\llbracket FMutex \rrbracket_{\Gamma}$. Test in pseuco.com. Do you think that $Mutex_2$ and $FMutex$ has the same behaviour?

7. Model a

- As acções do comboio caracterizam a sua localização em relação à passagem: aproxima-se (*approach*), entra (*in*) e sai (*exit*).
- A cancela pode estar baixada (*down*) ou levantada (*up*).
- O comboio e a cancela não comunicam directamente, apenas o controlador comunica com eles. O controlador deve garantir que quando o comboio entra a cancela está em baixo.

- (a) Descreve em CCS cada componente: comboio, cancela e controlador.
- (b) Escreve o processo do sistema que corresponda à execução paralela das várias componentes, indicando as acções em que devem sincronizar.
- (c) Use the CSS inference rules (mentioning the name of the rule used in each step) to obtain the reachable fragment of that process.

8. Consider:

$$\begin{aligned} C_0 &:= inc.C_1 \\ C_n &:= inc.C_{n+1} + dec.C_{n-1}, \text{ para } n \geq 1 \end{aligned}$$

e $SC := inc.(SC|dec.0)$.

- (a) Determine the initial fragment $\llbracket C_0 \rrbracket_{\Gamma}$. Argue that the LTS cannot be state-finite.
- (b) Determine the accessible fragment of LTS (and initial) $\llbracket SC \rrbracket_{\Gamma}$
- (c) Determine if the two LTS are isomorphic. Is there any similarity between them?